

SYSTEM DESIGN TRAINING PROGRAM

Day 5

Containers to a Secured, Meshed System — Final Day

Case Study: RetailOrderHub — deployed to AKS, meshed with Istio, and secured

Language: Java **Platform:** AKS, Istio, Azure Key Vault / Azure AD

Final day • Capstone presentations this afternoon

Today's Agenda

Four hours, from decomposed on paper to deployed, meshed, and secured

- | | | |
|----|-------------------------------------|---|
| 01 | Recap & Today's Flow | From service boundaries on paper to containers → AKS → Istio → Security |
| 02 | Kubernetes & AKS | Containers, Pods, Deployments, Services — and deploying RetailOrderHub |
| 03 | Service Mesh with Istio | Sidecars, mTLS, canary/blue-green routing, and distributed tracing |
| 04 | Security | Authentication, authorization, JWT validation, encryption, and vulnerability management |
| 05 | Hands-on Labs & Capstone | Deploy to AKS, a Service Mesh lab, a JWT security lab, and capstone presentations |

Today, We Make It Real

Yesterday we drew the boundaries. Today, we run them, connect them, and lock them down.

Yesterday: order-service, inventory-service, and payment-service, each with its own database — payment-service is CP, the product catalog is AP, inventory-service sparked real debate.



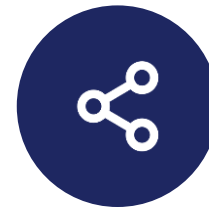
Containers

Package each service, consistently,
anywhere



AKS

Deploy, scale, and self-heal on
Kubernetes



Istio

Encrypt and observe every call,
uniformly



Security

Real authentication, authorization, and
secrets

This afternoon, you present the whole week's work as a capstone — HLD through security, all in one design.

Kubernetes Fundamentals

From VMs, to containers, to orchestration

Why containers, and why orchestrate them? A container packages code with everything it needs to run, consistently, anywhere. Kubernetes then automates deploying, scaling, and healing many containers across many machines.



Container

A packaged, runnable unit — RetailOrderHub's inventory-service image, for example



Pod

The smallest deployable unit in K8s — one or more tightly-coupled containers sharing storage and network



Deployment

Declares how many replicas of a Pod should run, and manages rolling updates



Service

A stable network endpoint that load-balances across a Deployment's Pods



ConfigMap / Secret

Externalized configuration and sensitive values, injected into Pods at runtime

Scaling, Self-Healing & AKS

What Kubernetes does automatically, and why AKS



Self-Healing & Auto-Scaling

- If a Pod crashes, Kubernetes restarts it automatically — no manual intervention. You'll trigger this yourself in today's lab.
- If a node fails, Pods are rescheduled onto healthy nodes.
- Horizontal Pod Autoscaling adds or removes replicas based on CPU/memory or custom metrics — this is Day 3's auto-scaling idea, native to the platform.



Azure Kubernetes Service (AKS)

- Azure manages the Kubernetes control plane — no cluster management overhead for the team.
- Integrates natively with Azure AD, Azure Key Vault, and Azure Monitor.
- Same kubectl and Kubernetes manifests you'd use anywhere — no lock-in to a proprietary API.

Deploying RetailOrderHub to AKS

Four backend services, four independent Deployments, fronted by one gateway

AKS Cluster — namespace: retailorderhub

api-gateway (LoadBalancer — public entry point)

Deployment: order-service

Pod (x1)

Service (ClusterIP)

orders (H2)

Deployment: inventory-service

Pod (x1)

Service (ClusterIP)

inventory (H2)

Deployment: payment-service

Pod (x1)

Service (ClusterIP)

— (stateless)

Each service scales, deploys, and fails independently — a crash in inventory-service no longer risks payment-service. replicas: 1 keeps today's self-healing demo easy to watch.

Service Mesh

Managing microservices communication with Istio

A service mesh handles service-to-service communication as infrastructure, not application code. A lightweight proxy (a sidecar) runs alongside every service instance and intercepts all network traffic in and out.



Traffic Management

Fine-grained routing, retries, and timeouts — configured outside the application, not hardcoded into it



mTLS Everywhere

Every service-to-service call is automatically encrypted and authenticated — without a single line of application code

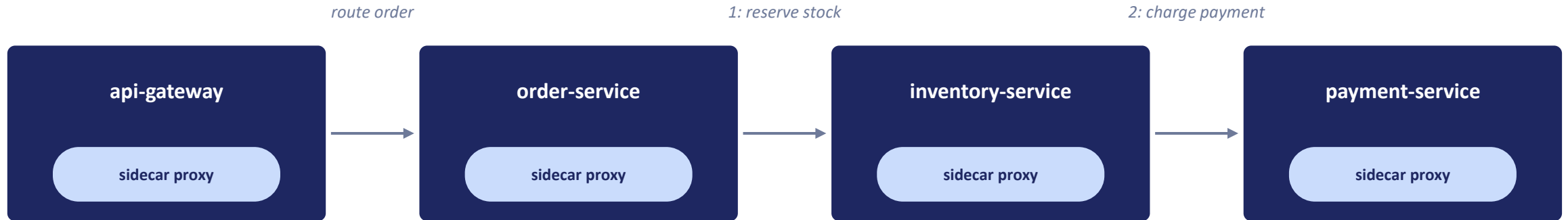


Observability

Every request is traced automatically across service boundaries — where Day 3's Circuit Breaker met one dependency, this covers all of them

Tracing a Request Through the Mesh

Placing an order, end to end, with Istio sidecars



Every arrow above is intercepted by a sidecar, automatically. The call is encrypted (mTLS), a retry policy applies if inventory-service is momentarily slow, and the whole trace — all three hops — shows up as one connected request in the observability dashboard.

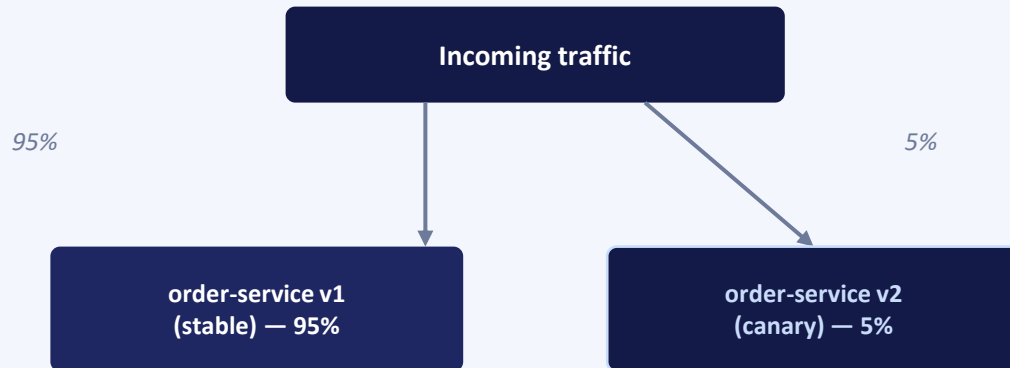


None of this touches any service's code. Compare this to Day 3's Circuit Breaker, which we added as a decorator around one interface. The mesh applies that same idea — resilience and observability without modifying business logic — to every call in the system at once.

Traffic Management in Practice

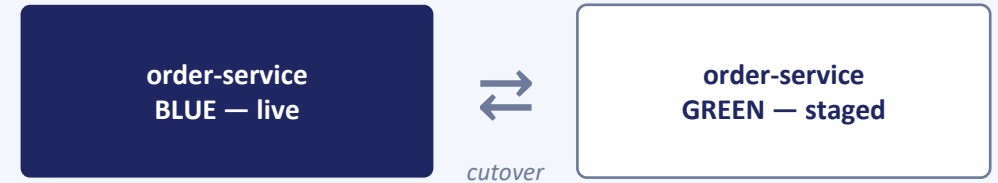
Canary releases and blue-green cutovers — routing rules, not application code

Canary Release



Shift traffic gradually as confidence grows — a VirtualService traffic weight, not a redeploy. Watch error rates on v2 before shifting more.

Blue-Green Cutover

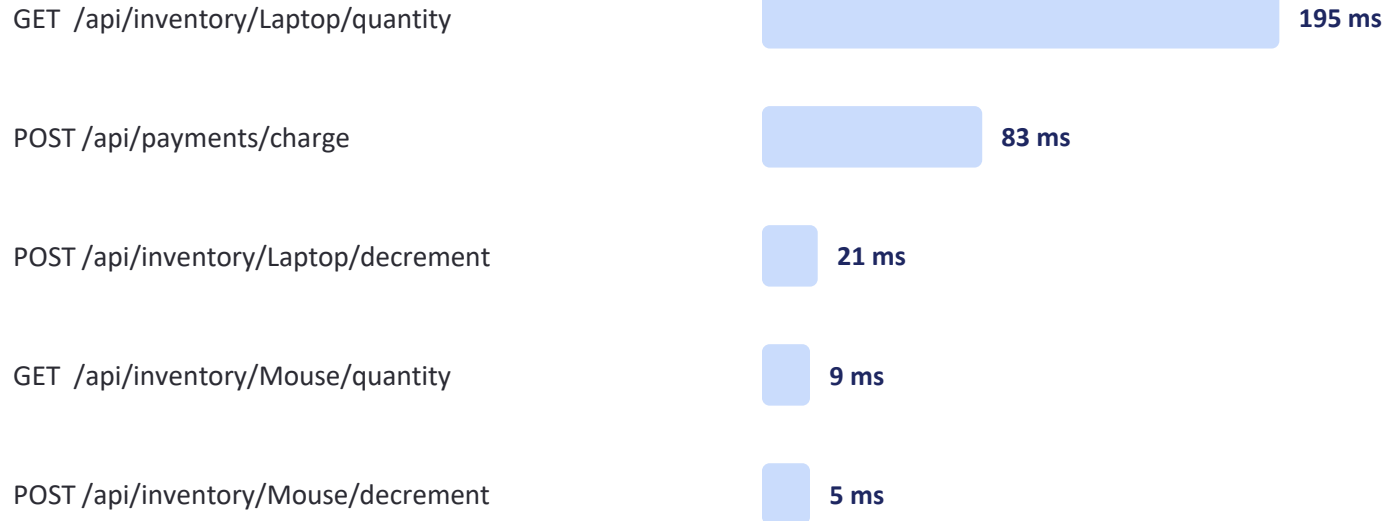


Test GREEN fully before it takes any real traffic, then flip all traffic at once — instant rollback is just flipping back.

Observability in Practice

One request, five hops, all connected — timings from your own lab run

POST /api/orders — 550 ms total (trace ID: f83d934c...)



This is the same trace you read by hand. You pulled these five calls out of the sidecar's own log in the mesh lab. A tracing backend (Jaeger/Zipkin) captures the same data automatically and correlates all five hops under one trace ID — this view instead of grepping logs.

Security

Authentication, authorization, encryption, and vulnerability management



Authentication & Authorization

Verify who's calling (Azure AD), then verify what they're allowed to do. Every service-to-service call should carry a verifiable identity — not just an open internal network.



Data Encryption

Encrypt data at rest (databases, backups) and in transit (mTLS, handled by the mesh). Secrets — connection strings, API keys — live in Azure Key Vault, never in code or config files.



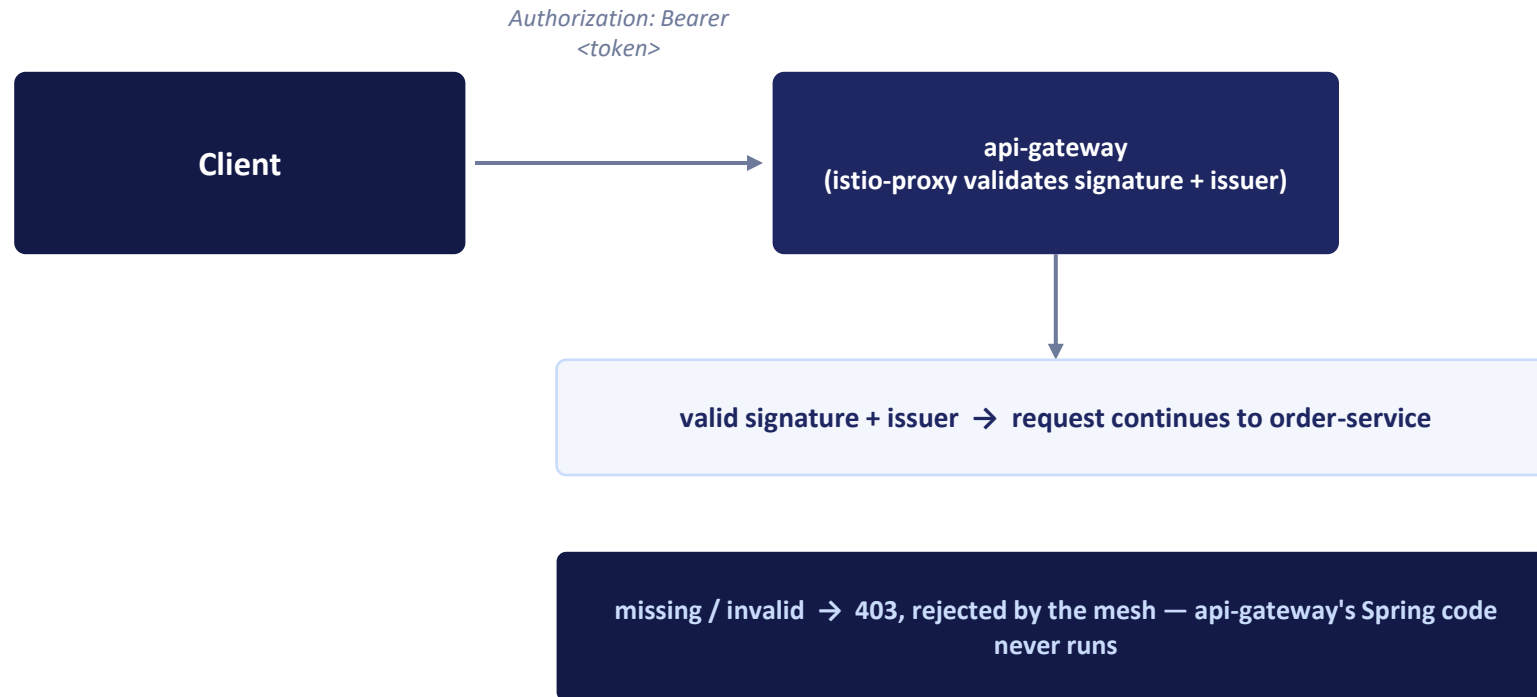
Vulnerability Management

Regularly scan dependencies and container images for known vulnerabilities. Recall Day 1's SonarCloud findings — this is that same discipline, applied continuously in production.

What's next: the upcoming lab enforces exactly this — Istio's `RequestAuthentication` and `AuthorizationPolicy` validate a signed JWT before any request reaches api-gateway's code, no Java touched. Separately: a shared API key never expires and can't be traced to a caller, and copying a secret from Key Vault by hand can drift out of sync on rotation — Managed Identity avoids that entirely.

Securing api-gateway: JWT Validation

The mesh checks the token before your code ever sees the request



Two Kubernetes objects, zero Java. RequestAuthentication validates the token's signature and issuer. AuthorizationPolicy decides whether a request without one gets in at all. Both are declarative YAML — the same kind of object as the Deployments and Services you already applied, not application code.

Hands-On Labs

Approximately 1.5 – 2 hours • All three labs continue on the same AKS cluster



Lab 1 — Deploy to AKS

Deploy RetailOrderHub's four backend services to AKS, then trigger a real failure and watch Kubernetes self-heal it.



Lab 2 — Service Mesh with Istio

Install Istio on the same cluster, restart the Deployments to pick up sidecars, and confirm every call is now encrypted and traceable.



Lab 3 — Securing api-gateway

Require a signed JWT on every request using Istio's RequestAuthentication and AuthorizationPolicy — two YAML files, zero Java changes.

Note on Lab 3: *the JWT check uses Istio's public test signing key so no identity provider needs to be stood up for class — a real system points RequestAuthentication at Azure AD (or similar) instead.*

Materials & Setup Checklist

What you'll need on hand before we start the labs



RetailOrderHub Project & Azure Access

The provided project zip (Dockerfiles + k8s manifests included), and an Azure subscription with rights to create a resource group, AKS cluster, and container registry



CLI Tools Installed

Azure CLI, kubectl, and istioctl on your machine — each lab's Prerequisites section has exact install/verify commands



JDK 17 & Maven

To build the service jars once before they go into containers — same as Day 2's Git workflow lab, no new setup



Capstone Presentation Template

HLD diagram, scaling notes, decomposition diagram, and security checklist

Capstone: RetailOrderHub, End to End

A design walkthrough and discussion — not a new build

Day 1

HLD & Design Quality

Present RetailOrderHub's high-level architecture and the design smells you found and fixed

Day 2

SOLID Refactor

Show how OrderManager became OrderService, InventoryService, and PaymentService

Day 3

Scaling & Resilience

Explain the load-balancing, caching, and Circuit Breaker decisions and why

Day 4

Sharding & Decomposition

Justify your service boundaries and your CAP trade-off for InventoryService

Day 5

Deployment & Security

Walk through the AKS deployment, the Istio service mesh, and your security controls

Course Wrap-Up

Five days, one system, fully covered — The RetailOrderHub Journey

Day 1: a monolith with a God Object, found through real analysis

Day 2: refactored through SOLID into clean, focused services

Day 3: scaled horizontally, cached, and made resilient

Day 4: database optimized, sharded, and decomposed

Day 5: deployed to AKS, meshed with Istio, and secured

All 8 Expectations — Covered

✓ HLD and LLD

✓ Implement design concepts

✓ Scaling application

✓ Resilience

✓ Circuit Breaker

✓ Sharding

✓ Service Mesh

✓ K8s

Every concept was grounded in one running system, from the first design smell on Day 1 to the last security control today. The patterns transfer — the next system you design won't be RetailOrderHub, but the questions you ask about it will be the same ones we asked all week.

Thank you for a great week.